

""Here is my writeups.""

Then,let's start.

Reserve

逆向工程入门指北 (345y! ! !)

用csdn上大佬分享的新版ida pro，打开下载的chall1

看到main函数部分

```
strcpy(s2, "moectf{C0ngr4tuLati0N_On_find1n9_your_1st_RE_f1aggggg!!!}");
```

结束。

flag:

moectf{C0ngr4tuLati0N_On_find1n9_your_1st_RE_f1aggggg!!!}

或许会有其他方法，但我这种应该挺简单

```
sunlight_tr4vel3r:sunlight_traveler ~/逆向工程入门指北 — in MoeCTF 2026 [12:32:31]
$ moectf{C0ngr4tuLati0N_On_find1n9_your_1st_RE_f1aggggg!!!}
[*] 正在提交: moectf{C0ngr4tuLati0N_On_find1n9_your_1st_RE_f1aggggg!!!}
已提交, 正在等待评测...
[+] 正确: Correct!
sunlight_tr4vel3r:sunlight_traveler ~/逆向工程入门指北 — in MoeCTF 2026 [12:32:40]
$
```

Assembly(汇编爱好者狂喜)

直接用记事本打开下载的chall2.txt

看向第一行，很显然，看到mov ecx,25，立刻记下：ecx寄存器目前储存的值：25

还有lea rsi,[byte 404000]，那我们看到404000：

byte_404000:

db 0x6d, 0x6f, 0x65, 0x63, 0x74, 0x66, 0x7b

db 0x41, 0x73, 0x73, 0x65, 0x6d, 0x62, 0x31, 0x79

db 0x5f, 0x4c, 0x34, 0x6e, 0x67, 0x75, 0x61, 0x67, 0x65, 0x5f

熟知16进制的朋友们都知道，0x开头是16进制的标志。

直接解ascii码，得到

往下看，看到loop段，这里在复制[rsi]到[r di]，再分别+1,记得我们之前记下的ecx=25吗，这一段每loop一次，ecx-1知道0结束。那么我们从404000按顺序抽25个字节。（正好25个）

按顺序解一下ascii码：moectf{Assemb1y_L4nguage_，显然是flag前半段了。

继续往下看

eax = 0x2a

再加0x16

再cmp比较与0x40的值。

如果你不能直接看出来，那么就算：0x2a转10进制= $2*16+10=42$

$0x16=16+6(10)=22(10)$

$42+22=64$

$0x40=4*16=64=eax$

对于jne指令，此时cmp的结果是zf=1,, 不跳转。

接着往下看

$ebx=0x10=16$

shl ebx,2, 相当于 $ebx*4=64$

与eax相等，同理不跳。

接着往下看

$ecx=0x39$

$ecx-=0x20$

$ecx=0x19>0x18$

对于jg指令，此时cmp的结果是zf=0且sf=of, 跳转。

那么我们查看401050的相关内容。

$ecx=15$ ，先记下

往下看401060的内容，我们立刻知道，401050-401060这段是从404020获取15个字节，每个分别与0x42异或。然后解ascii码。

计算：

原字节	异或 0x42	字符
0x73	0x31	1
0x11	0x53	s
0x1d	0x5f	_
0x21	0x63	c
0x2d	0x6f	o
0x72	0x30	0
0x2d	0x6f	o
0x72	0x30	0
0x0d	0x4f	o
0x2d	0x6f	o
0x2d	0x6f	o
0x2e	0x6c	l
0x63	0x21	!
0x63	0x21	!
0x3f	0x7d	}

1S_co0o0Oool!!!}很显然是flag右半边了。

那么结束

flag: moectf{Assembly_L4nguage_1S_co0o0Oool!!!}

```

sunlight_tr4vel3r:sunlight_traveler ~/Assembly — in MoeCTF 2026 [12:34:27]
$ moectf{Assembly_L4nguage_1S_co0o0Oool!!!}
[*] 正在提交: moectf{Assembly_L4nguage_1S_co0o0Oool!!!}
已提交, 正在等待评测...
[+] 正确: Correct!
sunlight_tr4vel3r:sunlight_traveler ~/Assembly — in MoeCTF 2026 [12:34:31]
$ _

```

bbxor

老样子, ida打开, 来看main函数, 直接伪代码启动。

牢玩家肯定很快就看懂了: 实际上这里的意思是: 程序要求输入36个10进制整数, 存入v4, 并与对应的 cipher值比对。当全部满足时, 输出yes

```

printf("Input: ");
for ( i = 0; i <= 35; ++i )
    __isoc99_scanf("%d", &v4[i]);

for ( j = 0; j <= 35; ++j )

```

```

{
    if ( (v4[j] ^ 0x66) != cipher[j] )
    {
        printf("no");
        return 0;
    }
}
printf("yes");
return 0;

```

我们直接双击这个cipher，看看跳转到哪

```

.data:0000000000404040 cipher      dd 0Bh, 9, 3, 5, 12h, 0, 1Dh, 24h, 52h, 15h, 0Fh, 5, 39h
.data:0000000000404074             dd 1Eh, 56h, 14h, 39h, 5, 0Eh, 7, 57h, 0Ah, 3, 8, 1, 3
.data:00000000004040A8             dd 39h, 15h, 9, 0Ah, 10h, 3, 2, 2 dup(47h), 1Bh

```

熟知16进制的牢玩家们肯定知道，h后缀是16进制的标志。

那么数据我们全得到了，不想手算，就写python脚本。

```

cipher = [
    0x0B, 0x09, 0x03, 0x05, 0x12, 0x00, 0x1D, 0x24,
    0x52, 0x15, 0x0F, 0x05, 0x39, 0x1E, 0x56, 0x14,
    0x39, 0x05, 0x0E, 0x07, 0x57, 0x0A, 0x03, 0x08,
    0x01, 0x03, 0x39, 0x15, 0x09, 0x0A, 0x10, 0x03,
    0x02, 0x47, 0x47, 0x1B
]

flag = ''.join(chr(c ^ 0x66) for c in cipher)
print(flag)

##### ##输出程序实际需要输入的十进制整数

nums = [c ^ 0x66 for c in cipher]
print(' '.join(map(str, nums)))

```

结果：moectf{B4sic_x0r_cha1lenge_solved!!}

109 111 101 99 116 102 123 66 52 115 105 99 95 120 48 114 95 99 104 97 49 108 101 110 103 101
95 115 111 108 118 101 100 33 33 125

答案已经显然，我们还是把下面这些数带入input检验，果然是yes

flag: moectf{B4sic_x0r_cha1lenge_solved!!}

```

sunlight_tr4vel3r:sunlight_traveler ~/bbxor — in MoeCTF 2026 [12:35:11]
$ moectf{B4sic_x0r_challenge_solved!!}
[*] 正在提交: moectf{B4sic_x0r_challenge_solved!!}
已提交, 正在等待评测...
[+] 正确: Correct!
sunlight_tr4vel3r:sunlight_traveler ~/bbxor — in MoeCTF 2026 [12:56:25]
$

```

反方向的RC4

你懂的，ida打开，查看main函数，伪代码启动。

```

int __fastcall main(int argc, const char **argv, const char **envp)
{
    _BYTE v4[32]; // [rsp+0h] [rbp-40h] BYREF
    char s[32]; // [rsp+20h] [rbp-20h] BYREF

    memset(s, 0, 0x15uLL);
    memset(v4, 0, 0x14uLL);
    printf("Input something: ");
    fgets(s, 21, _bss_start);
    init_cipher(v4);
    rc4_crypt(v4, 20LL, "Th1S_1s_secret~!", 16LL);
    puts("Done.");
    return 0;
}

```

看不出来啥，我们必须查看rc4_crypt以及init_cipher，去看看吧

rc4_crypt函数:

```

__int64 __fastcall rc4_crypt(__int64 a1, int a2, __int64 a3, int a4)
{
    __int64 result; // rax
    _BYTE v5[265]; // [rsp+18h] [rbp-120h]
    char v6; // [rsp+121h] [rbp-17h]
    char v7; // [rsp+122h] [rbp-16h]
    char v8; // [rsp+123h] [rbp-15h]
    unsigned int k; // [rsp+124h] [rbp-14h]
    int v10; // [rsp+128h] [rbp-10h]
    int j; // [rsp+12Ch] [rbp-Ch]
    int v12; // [rsp+130h] [rbp-8h]
    int i; // [rsp+134h] [rbp-4h]

    for ( i = 0; i <= 255; ++i )
        v5[i] = i;
    v12 = 0;
    for ( j = 0; j <= 255; ++j )
    {
        v12 = (unsigned __int8)(v5[j] + v12 + *(_BYTE *) (j % a4 + a3));
        v6 = v5[j];
        v5[j] = v5[v12];
        v5[v12] = v6;
    }
    v10 = 0;
    v12 = 0;
    for ( k = 0; ; ++k )
    {
        result = k;
        if ( (int)k >= a2 )
            break;
        v10 = (unsigned __int8)(v10 + 1);
        v12 = (unsigned __int8)(v5[v10] + v12);
        v8 = v5[v10];
        v5[v10] = v5[v12];
        v5[v12] = v8;
        v7 = v5[(unsigned __int8)(v5[v10] + v5[v12])];
        *(_BYTE *) ((int)k + a1) ^= v7;
    }
}

```

```

}
return result;
}

```

反正是个加密流程，实在不行直接复制粘贴给python(当然，语法需要调整)。现在，我们需要找密钥与密文。

双击一下这个Th1S_1s_secret~!，跳转到

```

.rodata:000000000402018      public g_key
.rodata:000000000402018      db 'Th1S_1s_secret~!',0 ; DATA XREF: main+79fo
.rodata:000000000402021      align 10h
.rodata:000000000402030      public g_cipher
.rodata:000000000402030      g_cipher      db 002h ; DATA XREF: init_cipher+27fo
.rodata:000000000402031      db 24h ; $
.rodata:000000000402032      db 24h ; $
.rodata:000000000402033      db 3
.rodata:000000000402034      db 0E3h
.rodata:000000000402035      db 0E4h
.rodata:000000000402036      db 41h ; A
.rodata:000000000402037      db 62h ; b
.rodata:000000000402038      db 11h
.rodata:000000000402039      db 0F3h
.rodata:00000000040203A      db 8Ah
.rodata:00000000040203B      db 28h ; (
.rodata:00000000040203C      db 0A0h
.rodata:00000000040203D      db 71h ; q
.rodata:00000000040203E      db 98h
.rodata:00000000040203F      db 0E5h
.rodata:000000000402040      db 27h ; '
.rodata:000000000402041      db 46h ; F
.rodata:000000000402042      db 19h
.rodata:000000000402043      db 0A9h
.rodata:000000000402044      ; const char format[]
.rodata:000000000402044      format      db 'Input something: ',0 ; DATA XREF: main+38fo
.rodata:000000000402056      ; const char s[]
.rodata:000000000402056      s      db 'Done.',0 ; DATA XREF: main+8Dfo
.rodata:000000000402056      _rodata      ends

```

高手都看出来了，g_key指的是密钥，g_cipher指的是密文。

这个g_cipher值可以直接复制粘贴了：b2 24 24 03 e3 b4 41 62 11 f3 8a 28 a0 71 9b be 27 46 19 a9

g_key也只要解一下ascii码：54 68 31 53 5f 31 73 5f 73 65 63 72 65 74 7e 21

那么我们着手编写python代码，或者上网抄一份，数值自己填。

```
def rc4_crypt(data: bytes, key: bytes) -> bytes:
```

```
    S = list(range(256))
```

```
    j = 0
```

```
    for i in range(256):
```

```
        j = (j + S[i] + key[i % len(key)]) & 0xFF
```

```
        S[i], S[j] = S[j], S[i]
```

```
    out = bytearray(data)
```

```
    i = 0
```

```
    j = 0
```

```
    for n in range(len(out)):
```

```
        i = (i + 1) & 0xFF
```

```
        j = (j + S[i]) & 0xFF
```

```
        S[i], S[j] = S[j], S[i]
```

```
        out[n] ^= S[(S[i] + S[j]) & 0xFF]
```

```
    return bytes(out)
```

```
key = bytes.fromhex("546831535f31735f7365637265747e21")
```

```
cipher = bytes.fromhex("b2242403e3b4416211f38a28a0719bbe274619a9")
```

```
plain = rc4_crypt(cipher, key)
```

```
print("ascii:", plain.decode('latin-1'))
```

运行即可。

flag: moectf{OH~Dyn4mic!}

```
sunlight_tr4vel3r:sunlight_traveler ~/反方向的 RC4 ————— in MoeCTF 2026 [13:00:34]
$ moectf{OH~Dyn4mic!}
[*] 正在提交: moectf{OH~Dyn4mic!}
已提交, 正在等待评测...
[+] 正确: Correct!
sunlight_tr4vel3r:sunlight_traveler ~/反方向的 RC4 ————— in MoeCTF 2026 [13:19:10]
```

Ultra Potato Xplosion

这次先读题目吧，显然提示我们是UPX加壳。那么牢办法暂时被ban了，我们需要先脱壳。

这里用wsl2.

```
upx -d chall8 -o unpacked
```

再用ida打开unpacked，老办法。

这下一眼可见Fnrr68qUFDSTA5tupNp8pgrFPJNesF3R6KNk

并且从input flag可知，我们在得到flag后可以来比对。正确就会输出correct，否则no

熟知base64的高手们知道，这个不可能是base64（长度不符，也没=）

那么可能是base58吗？相当有可能（无0、O、I、l、=）

解一下试试看

Fnrr68qUFDSTA5tupNp8pgrFPJNesF3R6KNk

输入格式 Base58

输出格式 String

字符集

UTF-8

Base58 编码

Base58 解码

复制

下载

清空

转换结果

moectf{EaSy_UPX_anD_Base58}

那么结束

flag: moectf{EaSy_UPX_anD_Base58}

```
sunlight_tr4vel3r:sunlight_traveler ~/Ultra Potato Xpl... ————— in MoeCTF 2026 [13:19:22]
$ moectf{EaSy_UPX_anD_Base58}
[*] 正在提交: moectf{EaSy_UPX_anD_Base58}
已提交, 正在等待评测...
[+] 正确: Correct!
sunlight_tr4vel3r:sunlight_traveler ~/Ultra Potato Xpl... ————— in MoeCTF 2026 [13:37:47]
```

奇怪的APP

再见了ida小弟，这个你打不开。

首先我们需要一个能分析.apk的工具

这里我使用jadx

```
package com.moectr.apkreverse;

import android.app.Activity;
import android.os.Bundle;

/* JADX INFO: loaded from: classes.dex */
public class MainActivity extends Activity {
    public static String 00o00o0() {
        return "RXYzcnNlXzFz";
    }

    @Override // android.app.Activity
    protected void onCreate(Bundle bundle) {
        super.onCreate(bundle);
    }
}
```

在MainActivity中，这个RXYzcnNlXzFz实在显眼，正好12长度，试试base64解码

Base64 编码/解码

将文本编码为 Base64 或将 Base64 解码回文本格式。

RXYzcnNlXzFz

字符编码: UTF-8

☒ 解码过滤非 Base64 字符

Base64 编码

Base64 解码

↕ 交换

清空

Ev3rse_1s

Ev3rse_1s

先放着。继续看下面的内容。

```
{
  "api": "local",
  "debug": false,
  "theme": "warmup",
  "cache_key": "Z2hUPyEhIX0=",
  "note": "Nothing interesting here... probably."
}
```

位于config.json，又是一段Z2hUPyEhIX0=，12长度还有=，base64启动

ghT?!!!}

右括号很爽，确定这个是最右边的内容。

继续往下。

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">APK Reverse Warmup</string>
    <string name="looks_like_a_theme_key">X2Z1Tl9yMQ==</string>
</resources>
```

位于strings.xml

依旧base64

_fuN_r1

是中间内容，不妨先拼一下试试。

Ev3rse_1s_fuN_r1ghT?!!!}

顺便猜一下，这个肯定是?? reverse is fun right

继续往下

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.moectf.apkreverse"
    android:versionCode="1"
    android:versionName="1.0">
    <uses-sdk
        android:minSdkVersion="21"
        android:targetSdkVersion="28"/>
    <uses-permission android:name="bW9LY3Rme0Fwa19S"/>
    <application android:label="APK Reverse Warmup">
        <activity
            android:name="com.moectf.apkreverse.MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
    </application>
</manifest>
```

位于AndroidManifest.xml

你懂的，解码去

moectf{Apk_R

好啦！结束。

flag: moectf{Apk_REv3rse_1s_fuN_r1ghT?!!!}

请你喝茶

老办法，你懂的。

这次信息不多，但是我们可以看到有个check_tea_part, 和check_xtea_part

查看相关函数。

在check_tea_part函数伪代码中，双击get_tea_key_address，来到对应函数，再双击tea_key，看到一些16进制值，并且下方看到tea_cipher和相邻的几个地址。不妨把值记下来，可能就是密钥，密文的值。

```
.rodata:0000000000402010 tea_key db 78h ; x ; DATA XREF: get_tea_key_address+810
.rodata:0000000000402011 db 56h ; V
.rodata:0000000000402012 db 34h ; 4
.rodata:0000000000402013 db 12h
.rodata:0000000000402014 | db 21h ; !
.rodata:0000000000402015 db 43h ; C
.rodata:0000000000402016 db 65h ; e
.rodata:0000000000402017 db 87h
.rodata:0000000000402018 db 68h ; h
.rodata:0000000000402019 db 24h ; $
.rodata:000000000040201A db 57h ; W
.rodata:000000000040201B db 13h
.rodata:000000000040201C db 57h ; W
.rodata:000000000040201D db 13h
.rodata:000000000040201E db 68h ; h
.rodata:000000000040201F db 24h ; $
.rodata:0000000000402020 xtea_key db 0EFh ; DATA XREF: get_xtea_key_address+810
.rodata:0000000000402021 db 0BEh
.rodata:0000000000402022 db 0ADh
.rodata:0000000000402023 db 0DEh
.rodata:0000000000402024 db 0BEh
.rodata:0000000000402025 db 0BAh
.rodata:0000000000402026 db 0FEh
.rodata:0000000000402027 db 0CAh
.rodata:0000000000402028 db 44h ; D
.rodata:0000000000402029 db 33h ; 3
.rodata:000000000040202A db 22h ; "
.rodata:000000000040202B db 11h
.rodata:000000000040202C db 88h
.rodata:000000000040202D db 77h ; w
.rodata:000000000040202E db 66h ; f
.rodata:000000000040202F db 55h ; U
.rodata:0000000000402030 tea_cipher dd 0B3E7E33Eh ; DATA XREF: check_tea_part+8E1r
.rodata:0000000000402034 dword_402034 dd 0B4114672h ; DATA XREF: check_tea_part+A21r
.rodata:0000000000402038 dword_402038 dd 8E088C0Ch ; DATA XREF: check_tea_part+B61r
.rodata:000000000040203C dword_40203C dd 14B2C329h ; DATA XREF: check_tea_part+CA1r
.rodata:0000000000402040 xtea_cipher dd 60EC68ABh ; DATA XREF: check_xtea_part+8E1r
.rodata:0000000000402044 dword_402044 dd 940251CEh ; DATA XREF: check_xtea_part+A21r
.rodata:0000000000402048 dword_402048 dd 0B427534Ch ; DATA XREF: check_xtea_part+B61r
.rodata:000000000040204C dword_40204C dd 0DF435416h ; DATA XREF: check_xtea_part+CA1r
.rodata:0000000000402050 aInputYourFlag db 'Input your flag: '.0
```

结合tea6solve，这些确实就是我们要找的值。密文直接抄。在网上简单学习db小端序解析算法，自己算一下，就能获得密钥。那么现在只剩下填解密函数了。

```
cipher1[4] = {

    0xB3E7E33E,

    0xB4114672,

    0x8E088C0C,

    0x14B2C329

};

key1[4] = {

    0x12345678,

    0x87654321,

    0x13572468,
```

```

        0x24681357

};

cipher2[4] = {

    0x60EC68AB,

    0x940251CE,

    0xB427534C,

    0xDF435416

};

key2[4] = {

    0xDEADBEEF,

    0xCAFEFEBABE,

    0x11223344,

    0x55667788

};

```

看tea_encrypt函数

```

__int64 __fastcall tea_encrypt(unsigned int *a1, _DWORD *a2)
{
    __int64 result; // rax
    unsigned int i; // [rsp+20h] [rbp-10h]
    int v4; // [rsp+24h] [rbp-Ch]
    unsigned int v5; // [rsp+28h] [rbp-8h]
    unsigned int v6; // [rsp+2Ch] [rbp-4h]

    v6 = *a1;
    v5 = a1[1];
    v4 = 0;
    for ( i = 0; i <= 0x1F; ++i )
    {
        v4 -= 1640531527;
        v6 += (v5 + v4) ^ (16 * v5 + *a2) ^ ((v5 >> 5) + a2[1]);
        v5 += (v6 + v4) ^ (16 * v6 + a2[2]) ^ ((v6 >> 5) + a2[3]);
    }
    *a1 = v6;
    result = v5;
    a1[1] = v5;
    return result;
}

```

把里面vx, ax代表啥找出来，往里填就行

```

{
    cipher[1] -= (cipher[0] + delta) ^ (16 * cipher[0] + key[2]) ^ ((cipher[0] >> 5) + key[3]);
    cipher[0] -= (cipher[1] + delta) ^ (16 * cipher[1] + key[0]) ^ ((cipher[1] >> 5) + key[1]);
    delta += 1640531527;
}

```

xtea_encrypt也差不多

```

{
    cipher[1] -= (((cipher[0] << 4) ^ (cipher[0] >> 5)) + cipher[0]) ^ (key[(delta >> 11) & 3] + delta);
    delta -= 0x9E3779B9;
    cipher[0] -= (((cipher[1] << 4) ^ (cipher[1] >> 5)) + cipher[1]) ^ (key[delta & 3] + delta);
}

```

那么运行，结束

flag: moectf{Wh4t_a_n1ce_cup_of_TEA!!}

```

sunlight_tr4vel3r:sunlight_traveler ~/请你喝茶 ————— in MoeCTF 2026 [13:57:16]
$ moectf{Wh4t_a_n1ce_cup_of_TEA!!}
[*] 正在提交: moectf{Wh4t_a_n1ce_cup_of_TEA!!}
已提交，正在等待评测...
[+] 正确: Correct!
sunlight_tr4vel3r:sunlight_traveler ~/请你喝茶 ————— in MoeCTF 2026 [14:21:43]

```

Tell Me No Lies

老办法。

在main函数中，我们看到一些端倪。

```

int __fastcall main(int argc, const char **argv, const char **envp)
{
    char v4[32]; // [rsp+0h] [rbp-50h] BYREF
    _BYTE s[32]; // [rsp+20h] [rbp-30h] BYREF
    unsigned int v6; // [rsp+40h] [rbp-10h]
    unsigned int v7; // [rsp+44h] [rbp-Ch]
    __int64 v8; // [rsp+48h] [rbp-8h]

    memset(s, 0, 0x15uLL);
    memset(v4, 0, sizeof(v4));
    puts("=== MOE Restricted Recovery Console ===");
    puts("[*] <mark>Checking debugger state...</mark>");
    v7 = <mark>IsDebuggerPresent();</mark>
    if ( v7 )
    {
        puts("[!] <mark>Debugger detected.</mark>");
        puts("[-] <mark>Recovery console terminated</mark>.");
        <mark>return 1</mark>;
    }
    else
    {
        puts("[*] Verifying operator token...");
        build_required_token(s);
        <mark> v8 = acquire_operator_token();</mark>
        <mark> v6 = compare_operator_token(v8, s);</mark>
        if ( v6 )
        {
            puts("[!] <mark>Operator token mismatch.</mark>");
            puts("[-] <mark>Recovery console terminated.</mark>");
        }
    }
}

```

```

        <mark>return 1</mark>;
    }
    else
    {
        puts("[*] Authorization accepted.");
        puts("[*] Starting multilayer recovery engine...");
        if ( (unsigned int)self_driven_decrypt(v7, v6, v4) )
        {
            puts("[+] Recovery complete.");
            printf("[+] %s\n", v4);
            memset(v4, 0, sizeof(v4));
            return 0;
        }
        else
        {
            puts("[!] Recovery state corrupted.");
            puts("[-] No valid record recovered.");
            return 1;
        }
    }
}
}
}

```

debugger detect, 反调试。

但是再看看题目的信息：

藏起你身后的目光，让我相信此刻无人注视：IsDebuggerPresent()

再让那场注定失败的校验归于通过：compare_operator_token()

我便会把沉睡的秘密亲口告诉你：self_driven_decrypt()

再看看那个return 1

如果强行return 0，会怎么样？

现在我们需要更多信息。

wsl, 启动！

```

(base) sun11ght_tr4ve13r@s1eepymas07:/mnt/d$ file chall10
chall10: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically
linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, not
stripped
(base) sun11ght_tr4ve13r@s1eepymas07:/mnt/d$ checksec --file=chall10
RELRO           STACK CANARY      NX            PIE            RPATH
RUNPATH         Symbols      FORTIFY Fortified      Fortifiable    FILE
Partial RELRO   No canary found  NX enabled    No PIE          No RPATH      No
RUNPATH        59 Symbols      No            0              3chall10

```

关键：

- **not stripped**: 符号表完整，函数名一个都没少，IDA 打开直接 F5 就能读；
- **No PIE**: 基址固定 0x400000，所有地址可以写死，下断点、改内存都毫无障碍。

先跑一下试试

```
(base) sun11ght_tr4vel3r@Sleepymas07:/mnt/d$ ./chal110
=== MOE Restricted Recovery Console ===
[*] Checking debugger state...
[*] Verifying operator token...
[!] Operator token mismatch.
[-] Recovery console terminated.
```

那么, gdb动态调试。

现在先在ida中看看它们的地址。

```
.text:0000000000401256 IsDebuggerPresent proc near          ; CODE XREF: main+4C↓p
.text:0000000000401256
.text:0000000000401256 s1              = byte ptr -110h
.text:0000000000401256 var_106          = byte ptr -106h
.text:0000000000401256 stream          = qword ptr -10h
.text:0000000000401256 var_8           = qword ptr -8
.text:0000000000401256
.text:0000000000401256 ; __unwind {
✓.text:0000000000401256          endbr64
```

IsDebuggerPresent:0x401256

```
.text:00000000004014D6
.text:00000000004014D6 ; ===== S U B R O U T I N E =====
.text:00000000004014D6 ; Attributes: bp-based frame
.text:00000000004014D6
.text:00000000004014D6 ; __int64 __fastcall compare_operator_token(_QWORD, _QWORD)
.text:00000000004014D6 compare_operator_token proc near          ; CODE XREF: main+A6↓p
.text:00000000004014D6
.text:00000000004014D6 s2              = qword ptr -10h
.text:00000000004014D6 s1              = qword ptr -8
.text:00000000004014D6
.text:00000000004014D6 ; __unwind {
✓.text:00000000004014D6          endbr64
```

compare_operator_token:4014D6

```
.text:0000000000401E55 loc_401E55:          ; CODE XREF: main+B3↑j
.text:0000000000401E55          mov     edi, offset aAuthorizationA ; " [*] Authorization accepted."
.text:0000000000401E5A          call    _puts
.text:0000000000401E5F          mov     edi, offset aStartingMultil ; " [*] Starting multilayer recovery engine"...
.text:0000000000401E64          call    _puts
.text:0000000000401E69          mov     eax, [rbp+var_10]
.text:0000000000401E6C          mov     ecx, eax
.text:0000000000401E6E          mov     eax, [rbp+var_C]
.text:0000000000401E71          lea     rdx, [rbp+var_50]
.text:0000000000401E75          mov     esi, ecx
.text:0000000000401E77          mov     edi, eax
.text:0000000000401E79          call    self_driven_decrypt
✓.text:0000000000401E7E          test    eax, eax
```

self_driven_decrypt:401e79

现在写gdb脚本

```
set pagination off
set confirm off

break *0x401256
commands
    silent
    return (int)0
continue
end

break *0x4014d6
commands
    silent
    return (int)0
```

```
    continue
end

break *0x401e7e
commands
    silent
    printf "\n"
    continue
end

run
```

```
(base) sunlight_tr4vel3r@sleepymas07:/mnt/d$ nano wit.gdb
(base) sunlight_tr4vel3r@sleepymas07:/mnt/d$ gdb -nx -q -batch -x wit.gdb
./chall10
Breakpoint 1 at 0x401256
Breakpoint 2 at 0x4014d6
Breakpoint 3 at 0x401e7e

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
=== MOE Restricted Recovery Console ===
[*] Checking debugger state...
[*] Verifying operator token...
[*] Authorization accepted.
[*] Starting multilayer recovery engine...

[+] Recovery complete.
[+] moectf{Wow~h0w_do_Y0u_f1nd_IT?}
[Inferior 1 (process 1944) exited normally]
```

结束。完美。

flag: moectf{Wow~h0w_do_Y0u_f1nd_IT?}

```
sunlight_tr4vel3r:sunlight_traveler ~/Tell Me No Lies — in MoeCTF 2026 [14:22:13]
$ moectf{Wow~h0w_do_Y0u_f1nd_IT?}
[*] 正在提交: moectf{Wow~h0w_do_Y0u_f1nd_IT?}
已提交, 正在等待评测...
[+] 正确: Correct!
sunlight_tr4vel3r:sunlight_traveler ~/Tell Me No Lies — in MoeCTF 2026 [15:48:53]
```

成都好室友

#牢办法又没有用了, sad

竟然打不开, 查查成分

```
(base) sunlight_tr4vel3r@sleepymas07:/mnt/d$ xxd -g1 -l 32 chall15.pyc
00000000: 6f 0d 0d 0a 00 00 00 00 62 f5 49 6a 0b 3d 00 00  o.....b.Ij.=..
00000010: e3 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....
```

```
(base) sun11ght_tr4vel3r@Sleepymas07:/mnt/d$ file chall15.pyc
chall15.pyc: Byte-compiled Python module for CPython 3.10 (magic: 3439),
timestamp-based, .py timestamp: Sun Jul  5 06:10:42 2026 UTC, .py size: 15627
bytes
```

文件头没毛病: 6f 0d 0d 0a

版本是python3.10, 先创建虚拟环境

```
conda create -n py310 python=3.10 -y
```

```
conda activate py310
```

然后运行看看

```
(py310) sun11ght_tr4vel3r@Sleepymas07:/mnt/d$ python3.10 chall15.pyc
Successfully decode finished: result.png
Image data size: 11116 bytes.
```

看看result.png



观察右下角, 图片被反色!



反色一波，然后seedream ai补全



扫码得到结果，结束

flag: moectf{PYC_p1us_QRc0dE_M1ssing_c0rN3r}

```
sunlight_tr4vel3r:sunlight_traveler ~/成都好室友 — in MoeCTF 2026 [16:21:44]
$ moectf{PYC_p1us_QRc0dE_M1ssing_c0rN3r}
[*] 正在提交: moectf{PYC_p1us_QRc0dE_M1ssing_c0rN3r}
已提交，正在等待评测...
[+] 正确: Correct!
sunlight_tr4vel3r:sunlight_traveler ~/成都好室友 — in MoeCTF 2026 [16:32:09]
$ _
```

让我们说中文

#还是老办法用着爽。

```

__int64 __fastcall main(int a1, char **a2, char **a3)
{
    size_t v3; // rax
    const char *v4; // rax
    char *v5; // rbx
    char v7[4104]; // [rsp+0h] [rbp-1008h] BYREF

    __printf_chk(2LL, "Input cipher: ", a3);
    if ( !fgets(v7, 4096, stdin) )
    {
        puts("Input error.");
        return 1LL;
    }
    v7[strcspn(v7, "\r\n")] = 0;
    v3 = strlen(v7);
    v4 = (const char *)sub_401480(v7, v3);
    v5 = (char *)v4;
    if ( !v4 )
    {
        puts("Memory error.");
        return 1LL;
    }
    if ( !strcmp(
        v4,
        "KwC2gtPmKwn2NwTyKwGiltv0KwOiuqTvKw43utP2KwOxgttPmKwC2gtPmKwn21wUiKwIxnvtzKwn21Ta5Kw42KTP0KwIiNtt0Kw42NRT3KwM2g4"
        "JaKwn3NTarKwIxu5OrKw42NRT3KwOxKwUrKwIxu5OrKw42NRT3KwIxKtP3Kwn3N4PjKwIiK5T0Kw431RUxKwIiNtt0Kw42NRT3Kw42NwOxKwOi"
        "KTPzKwKxg50xKw42g5TpKwDx14JrKwIxu5OrKw42NRT3KwIxKtP3KwM2g4JaKwC2g4P3KwDxKwOaKw42NwMqKwIxu5OrKw42NRT3KwIiN5TzKw"
        "Kxg50xKw42g5TpKwDx14JrKwIiNtt0Kw42NRT3KwOxK4JaKwKxg50tKwGx1wU5Kwn3N4PjKwIiK5T0KwT31TPzKwn3N4PjKwIiK5T0KwKxg5Mq"
        "KwIxu5OrKw42NRT3Kw431vPjKwIxu5OrKw42NRT3KwM2g4JaKwn3NTarKw42KTPqKwn21Ta5Kw42KTP0" ) )
    {
        puts("Correct!");
    }
    else
    {
        puts("Wrong!");
        free(v5);
        return 0LL;
    }
}

```

这串乱码很可疑，拿去base64解一下。按照题目给的信息，分别尝试UTF-8和GBK

结果不看了，没那么简单，再去找找看。

```

.rodata:000000000402013 aInputError db 'Input error.',0 ; DATA XREF: main:loc_4011DEf0
.rodata:000000000402020 ; const char reject[]
.rodata:000000000402020 reject db 0Dh,0Ah,0 ; DATA XREF: main+40f0
.rodata:000000000402023 ; const char aMemoryError[]
.rodata:000000000402023 aMemoryError db 'Memory error.',0 ; DATA XREF: main:loc_4011F1f0
.rodata:000000000402031 ; const char aCorrect[]
.rodata:000000000402031 aCorrect db 'Correct!',0 ; DATA XREF: main:loc_4011D0f0
.rodata:00000000040203A ; const char s[]
.rodata:00000000040203A s db 'Wrong!',0 ; DATA XREF: main+81f0
.rodata:000000000402041 align 20h
.rodata:000000000402060 ; const char s2[]
.rodata:000000000402060 s2 db 'KwC2gtPmKwn2NwTyKwGiltv0KwOiuqTvKw43utP2KwOxgttPmKwC2gtPmKwn21wUiK'
.rodata:000000000402060 ; DATA XREF: main+6Ef0
.rodata:0000000004020A1 db 'wIxnvtzKwn21Ta5Kw42KTP0KwIiNtt0Kw42NRT3KwM2g4JaKwn3NTarKwIxu5OrKw'
.rodata:0000000004020E2 db '42NRT3KwOxKwUrKwIxu5OrKw42NRT3KwIxKtP3Kwn3N4PjKwIiK5T0Kw431RUxKwI'
.rodata:000000000402123 db 'iNtt0Kw42NRT3Kw42NwOxKwOikTPzKwKxg50xKw42g5TpKwDx14JrKwIxu5OrKw42'
.rodata:000000000402164 db 'NRT3KwIxKtP3KwM2g4JaKwC2g4P3KwDxKwOaKw42NwMqKwIxu5OrKw42NRT3KwIiN'
.rodata:0000000004021A5 db '5TzKwKxg50xKw42g5TpKwDx14JrKwIiNtt0Kw42NRT3KwOxK4JaKwKxg50tKwGx1w'
.rodata:0000000004021E6 db 'U5Kwn3N4PjKwIiK5T0KwT31TPzKwn3N4PjKwIiK5T0KwKxg5MqKwIxu5OrKw42NRT'
.rodata:000000000402227 db '3Kw431vPjKwIxu5OrKw42NRT3KwM2g4JaKwn3NTarKw42KTPqKwn21Ta5Kw42KTP0'
.rodata:000000000402268 db 0
.rodata:000000000402269 align 20h
.rodata:000000000402280 aSix5starspjyoun db 'Six5starsPJYouNgLLkEw4Ik1nGABCDHFHQRTUVWXZbcdefghjmqpvzy0236789+/',0
.rodata:000000000402280 ; DATA XREF: sub_401480+23f0
.rodata:0000000004022C1 align 20h
.rodata:0000000004022E0 aAbcdefghijklmn db 'ABCDEFGHijklmnopQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/',0
.rodata:0000000004022E0 ; DATA XREF: sub_4012F0+48f0
.rodata:0000000004022E0 ; sub_401480+1Cf0
.rodata:0000000004022E0 _rodata ends

```

这块内容有意思，我们看到下面两行，ABCD....显然是标准base64，那上面的Six5star...是怎么回事呢？难道是自定义映射表？

现在就动手试试看。

得到

```

RTc4OEIxBRTY4NTk1RTZBQ0E3RThBMzk0RTU5MEI4RThCOEEyRTc4OEIxBRTY4QTIBRTVCN0E2RTY4Q
kFDRTU4RkI3RTVBNEE3RTU4Njk5RTg4OUJFRTY5NkFGRTVCMDhGRTU4Njk5RThCRTIGRTVCMDhGR
TU4Njk5RTVCREI5RTY5NUIwRTVBRDk3RTU5QjICRTVBNEE3RTU4Njk5RTU4NThCRThBRki2RTRCOD
hCRTU4ODkyRTdCQUJGRTVCMdHGRU4Njk5RTVCREI5RTg4OUJFRtC4OUI5RTdCRThFRTU4NTgzRT
VCMDhGRTU4Njk5RTVBNDk2RTRCODhCRTU4ODkyRTdCQUJGRTVBNEE3RTU4Njk5RThCRUJFRTRC
ODhERTZCQTIDRTY5NUIwRTVBRDk3RTk5QkI2RTY5NUIwRTVBRDk3RTRCODgzRTVCMDhGRTU4Njk
5RTU5Q0lwRTVCMDhGRTU4Njk5RTg4OUJFRTY5NkFGRTU4RkIzRTY4QkFDRTU4RkI3

```

base64? 试试看

E788B1E68595E6ACA7E8A394E590B8E8B8A2E788B1E68A9AE5B7A6E68BACE58FB7E5A4A7E58699E889BEE696AFE5B08FE58699E8BE9FE5B08FE58699E5BDB9E695B0E5AD97E59B9BE5A4A7E58699E5858BE8AFB6E4B88BE58892E7BABFE5B08FE58699E5BDB9E889BEE789B9E7BE8EE58583E5B08FE58699E5A496E4B88BE58892E7BABFE5A4A7E58699E8BEBEE4B88DE6BA9CE695B0E5AD97E99BB6E695B0E5AD97E4B883E5B08FE58699E59CB0E5B08FE58699E889BEE696AFE58FB3E68BACE58FB7

高手都看出来，6个6个一组。

E788B1/E68595/E6ACA7/E8A394/E590B8/E8B8A2/E788B1/E68A9A/E5B7A6/E68BAC/E58FB7/E5A4A7/E58699/E889BE/E696AF/E5B08F/E58699/E8BE9F/E5B08F/E58699/E5BDB9/E695B0/E5AD97/E59B9B/E5A4A7/E58699/E5858B/E8AFB6/E4B88B/E58892/E7BABF/E5B08F/E58699/E5BDB9/E889BE/E789B9/E7BE8E/E58583/E5B08F/E58699/E5A496/E4B88B//E58892/E7BABF/E5A4A7/E58699/E8BEBE/E4B88D/E6BA9C/E695B0/E5AD97/E99BB6/E695B0/E5AD97/E4B883/E5B08F/E58699/E59CB0/E5B08F/E58699/E889BE/E696AF/E58FB3/E68BAC/E58FB7

按照题目的提示，解成unicode形式

Unicode和UTF编码转换

输入:

对应字符:
爱慕欧裔吸踢爱抚左括号大写艾斯小写辟小写役数字四大写克诶下划线小写役艾特美元小写外下划线大写达不溜数字零数字七小写地小写艾斯右括号

聪明人知道，已经结束了。

#沙威玛，哦沙威玛~~

flag: moectf{Spe4K_e@\$y_W07ds}

```
sunlight_tr4vel3r:sunlight_traveler ~/让我们说中文 ————— in MoeCTF 2026 [17:05:57]
$ moectf{Spe4K_e@$y_W07ds}
[*] 正在提交: moectf{Spe4K_e@$y_W07ds}
已提交，正在等待评测...
[+] 正确: Correct!
sunlight_tr4vel3r:sunlight_traveler ~/让我们说中文 ————— in MoeCTF 2026 [17:06:01]
```

Mewtype

#牢办法又区了

先打开来看看，复制粘贴出来 (Ctrl+Shift+End)

```
exit(1 - (lambda myk43: (lambda myk7: lambda vol8: lambda myk9: myk7(vol8)(myk9))((lambda myk17: (lambda myk3: lambda vol4: myk3) if myk17 else lambda myk5: lambda vol6: vol6)
(len(myk43) == 54))((lambda: (lambda myk13: lambda vol14: myk13(vol14)(myk13))((lambda myk19: (lambda myk17: (lambda myk3: lambda vol4: myk3) if myk17 else lambda myk5: lambda vol6: vol6)((myk19 * myk19 * myk19 - myk19) % 6 == 0))(len(myk43))((lambda myk13: lambda vol14: myk13(vol14)(myk13))((lambda vol0: (lambda myk1: myk1(myk1))((lambda vol2: vol0(lambda a: vol2(vol2)(a))))(lambda myk39: lambda vol40: lambda myk41: lambda vol42: (lambda myk7: lambda vol8: lambda myk9: myk7(vol8)(myk9))((lambda myk17: (lambda myk3: lambda vol4: myk3) if myk17 else lambda myk5: lambda vol6: vol6)(myk41 >= len(vol40) - 1))((lambda: lambda myk3: lambda vol4: myk3)(lambda: (lambda myk13: lambda vol14: myk13(vol14)(myk13))((lambda myk7: lambda vol8: lambda myk9: myk7(vol8)(myk9))((lambda vol18: (lambda myk15: lambda
```

```

vol16: myk15((lambda vol10: lambda myk11: lambda vol12: vol10(vol12)(myk11))(vol16))(vol16))
((lambda myk17: (lambda myk3: lambda vol4: myk3) if myk17 else lambda myk5: lambda vol6:
vol6)((vol18 * vol18 + vol18) % 2 == 0))(lambda myk5: lambda vol6: vol6))(myk41))(lambda: (lambda
vol22: lambda myk23: lambda vol24: lambda myk25: lambda vol26: (lambda myk7: lambda vol8:
lambda myk9: myk7(vol8)(myk9))((lambda vol20: (lambda myk17: (lambda myk3: lambda vol4:
myk3) if myk17 else lambda myk5: lambda vol6: vol6)(vol20 == 0))(myk25))(lambda: (lambda
myk17: (lambda myk3: lambda vol4: myk3) if myk17 else lambda myk5: lambda vol6: vol6)(vol22 +
myk23 + vol24 * vol24 == vol26))((lambda myk7: lambda vol8: lambda myk9: myk7(vol8)(myk9))
((lambda myk21: (lambda myk17: (lambda myk3: lambda vol4: myk3) if myk17 else lambda myk5:
lambda vol6: vol6)(myk21 == 1))(myk25))(lambda: (lambda myk17: (lambda myk3: lambda vol4:
myk3) if myk17 else lambda myk5: lambda vol6: vol6)(vol22 * myk23 + vol24 == vol26))(lambda:
(lambda myk17: (lambda myk3: lambda vol4: myk3) if myk17 else lambda myk5: lambda vol6:
vol6)(vol22 * vol22 + myk23 * myk23 + vol24 * vol24 * vol24 == vol26))))((ord(vol40[myk41]))
(ord(vol40[myk41 + 1]))(ord(vol40[(myk41 + 17) % len(vol40)])))(vol42)((10111, 11290, 137705,
13834, 12017, 882822, 2861, 6952, 148686, 9290, 9962, 153274, 13069, 9595, 871218, 9834,
10851, 1584472, 9149, 5765, 159511, 9289, 5093, 126222, 11887, 4921, 152566, 13114, 6109,
140607, 13366, 9447, 156018, 9712, 11045, 1795540, 15852, 4864, 1389923, 10278, 5369,
1586542, 10519, 5995, 1281695, 4298, 5674, 963368, 10232, 5085, 880083, 10095, 15119)[myk41]
^ 90))(lambda: (lambda vol22: lambda myk23: lambda vol24: lambda myk25: lambda vol26:
(lambda myk7: lambda vol8: lambda myk9: myk7(vol8)(myk9))((lambda vol20: (lambda myk17:
(lambda myk3: lambda vol4: myk3) if myk17 else lambda myk5: lambda vol6: vol6)(vol20 == 0))
(myk25))(lambda: (lambda myk17: (lambda myk3: lambda vol4: myk3) if myk17 else lambda myk5:
lambda vol6: vol6)(vol22 + myk23 + vol24 * vol24 == vol26))((lambda myk7: lambda vol8: lambda
myk9: myk7(vol8)(myk9))((lambda myk21: (lambda myk17: (lambda myk3: lambda vol4: myk3) if
myk17 else lambda myk5: lambda vol6: vol6)(myk21 == 1))(myk25))(lambda: (lambda myk17:
(lambda myk3: lambda vol4: myk3) if myk17 else lambda myk5: lambda vol6: vol6)(vol22 * myk23
+ vol24 == vol26))(lambda: (lambda myk17: (lambda myk3: lambda vol4: myk3) if myk17 else
lambda myk5: lambda vol6: vol6)(vol22 * vol22 + myk23 * myk23 + vol24 * vol24 * vol24 ==
vol26))))((ord(vol40[myk41 + 1]))(ord(vol40[myk41]))(ord(vol40[(myk41 + 17) % len(vol40)])))(vol42)
((10111, 11290, 137705, 13834, 12017, 882822, 2861, 6952, 148686, 9290, 9962, 153274, 13069,
9595, 871218, 9834, 10851, 1584472, 9149, 5765, 159511, 9289, 5093, 126222, 11887, 4921,
152566, 13114, 6109, 140607, 13366, 9447, 156018, 9712, 11045, 1795540, 15852, 4864, 1389923,
10278, 5369, 1586542, 10519, 5995, 1281695, 4298, 5674, 963368, 10232, 5085, 880083, 10095,
15119)[myk41] ^ 90)))(myk39(vol40)(myk41 + 1)((vol42 + 1) % 3)))(myk43)(0)(0))(lambda myk13:
lambda vol14: myk13(vol14)(myk13))((lambda myk17: (lambda myk3: lambda vol4: myk3) if myk17
else lambda myk5: lambda vol6: vol6)((lambda vol0: (lambda myk1: myk1(myk1))(lambda vol2:
vol0(lambda a: vol2(vol2)(a))))(lambda myk27: lambda vol28: lambda myk29: 0 if myk29 ==
len(vol28) else ord(vol28[myk29]) + myk27(vol28)(myk29 + 1))(myk43)(0) == 5138))((lambda myk13:
lambda vol14: myk13(vol14)(myk13))((lambda myk17: (lambda myk3: lambda vol4: myk3) if myk17
else lambda myk5: lambda vol6: vol6)((lambda vol0: (lambda myk1: myk1(myk1))(lambda vol2:
vol0(lambda a: vol2(vol2)(a))))(lambda vol30: lambda myk31: lambda vol32: 0 if vol32 ==
len(myk31) else ord(myk31[vol32]) ^ vol30(myk31)(vol32 + 1))(myk43)(0) == 26))((lambda myk13:
lambda vol14: myk13(vol14)(myk13))((lambda myk17: (lambda myk3: lambda vol4: myk3) if myk17
else lambda myk5: lambda vol6: vol6)((lambda vol0: (lambda myk1: myk1(myk1))(lambda vol2:
vol0(lambda a: vol2(vol2)(a))))(lambda myk33: lambda vol34: lambda myk35: 0 if myk35 ==
len(vol34) else ord(vol34[myk35]) * ord(vol34[myk35]) + myk33(vol34)(myk35 + 1))(myk43)(0) ==
520600))((lambda myk17: (lambda myk3: lambda vol4: myk3) if myk17 else lambda myk5: lambda
vol6: vol6)((lambda vol0: (lambda myk1: myk1(myk1))(lambda vol2: vol0(lambda a: vol2(vol2)(a))))
(lambda vol36: lambda myk37: lambda vol38: 0 if vol38 == len(myk37) - 1 else ord(myk37[vol38]) *
ord(myk37[vol38 + 1]) + vol36(myk37)(vol38 + 1))(myk43)(0) == 467644)))))))(lambda: lambda myk5:
lambda vol6: vol6)))(input())(1)(0))

```

先别晕，有信息：10111, 11290, 137705, 13834, 12017, 882822, 2861, 6952, 148686, 9290, 9962, 153274, 13069, 9595, 871218, 9834, 10851, 1584472, 9149, 5765, 159511, 9289, 5093, 126222, 11887, 4921, 152566, 13114, 6109, 140607, 13366, 9447, 156018, 9712, 11045, 1795540, 15852, 4864, 1389923, 10278, 5369, 1586542, 10519, 5995, 1281695, 4298, 5674, 963368, 10232, 5085, 880083, 10095, 15119

查一下，经典的lambda混淆

[Lambda-revenge CTF逆向深度解析：从邱奇编码到约束求解的终极指南-CSDN博客](#)感谢csdn社区的支持。

lambda cond: lambda then: *lambda* else: cond(then)(else)这个相当于if

(lambda myk3: lambda vol4: myk3)这个是True

(lambda myk5: lambda vol6: vol6)这个是False

眼尖的都看到了：

len(myk43) == 54

即字符串输入长度（flag长度）应为54。

继续分析，看一下外层。

exit(1-(lambda myk43:.....)(input()))(1)(0))

查阅资料得知：这个exit(1-p)，如果p=1，就正常退出，否则报错退出。

那么我们来拆解p: (lambda myk43:.....)(input()))(1)(0))

我们的input()值先传给了myk43。

然后查阅资料知道后面的1，0不影响逻辑，那就放着。

这边高亮了一些数值计算。

有异或计算，与26比对。

经过分析（已经瞎了），找到了三种计算分支，由一个数值%3为0，1，2开始分支。

翻译一下。

```
ord(vol40[myk41])           # 第 i 个字符
ord(vol40[myk41 + 1])       # 第 i+1 个字符
ord(vol40[(myk41 + 17) % len(vol40)]) # 第 (i+17) % 54 个字符
```

这段是循环，执行53次

```
def check(s):
    if len(s) != 54:
        return False
    arr = [10111, 11290, 137705, 13834, 12017, 882822, 2861, 6952, 148686,
           9290, 9962, 153274, 13069, 9595, 871218, 9834, 10851, 1584472,
           9149, 5765, 159511, 9289, 5093, 126222, 11887, 4921, 152566,
           13114, 6109, 140607, 13366, 9447, 156018, 9712, 11045, 1795540,
           15852, 4864, 1389923, 10278, 5369, 1586542, 10519, 5995, 1281695,
           4298, 5674, 963368, 10232, 5085, 880083, 10095, 15119]
    for i in range(53):
        a = ord(s[i])
        b = ord(s[i+1])
        c = ord(s[(i+17) % 54])
```

```

    target = arr[i] ^ 90
    if i % 3 == 0:
        if a + b + c*c != target:
            return False
    elif i % 3 == 1:
        if a * b + c != target:
            return False
    else:
        if a*a + b*b + c*c*c != target:
            return False
    if sum(ord(ch) for ch in s) != 5138:
        return False
    xor_sum = 0
    for ch in s:
        xor_sum ^= ord(ch)
    if xor_sum != 26:
        return False
    if sum(ord(ch)**2 for ch in s) != 520600:
        return False
    if sum(ord(s[i]) * ord(s[i+1]) for i in range(53)) != 467644:
        return False
    return True

```

这就是这段代码的逻辑。

我们只需要逆向这一段。

写个python脚本吧。

用z3

```

from z3 import *
arr = [
    10111, 11290, 137705, 13834, 12017, 882822, 2861, 6952, 148686,
    9290, 9962, 153274, 13069, 9595, 871218, 9834, 10851, 1584472,
    9149, 5765, 159511, 9289, 5093, 126222, 11887, 4921, 152566,
    13114, 6109, 140607, 13366, 9447, 156018, 9712, 11045, 1795540,
    15852, 4864, 1389923, 10278, 5369, 1586542, 10519, 5995, 1281695,
    4298, 5674, 963368, 10232, 5085, 880083, 10095, 15119
]
s = [Int(f's{i}') for i in range(54)]
sol = Solver()
for x in s:
    sol.add(x >= 32, x <= 126)
for i in range(53):
    a, b = s[i], s[i + 1]
    c = s[(i + 17) % 54]
    target = arr[i] ^ 90
    if i % 3 == 0:
        sol.add(a + b + c * c == target)
    elif i % 3 == 1:
        sol.add(a * b + c == target)
    else:
        sol.add(a * a + b * b + c * c * c == target)
sol.add(Sum(s) == 5138)
sol.add(Sum([x * x for x in s]) == 520600)
sol.add(Sum([s[i] * s[i + 1] for i in range(53)]) == 467644)
m = sol.model()

```

```
flag = ''.join(chr(m[x].as_long()) for x in s)
print(flag)
```

运行，完工。

flag:

moectf{l@mbd4_c@lcu1us_4r3_h4rd_but_z3_s0lv3r_1s_3asy}

```
sunlight_tr4vel3r:sunlight_traveler ~/Mewtype — in MoeCTF 2026 [17:10:36]
$ moectf{l@mbd4_c@lcu1us_4r3_h4rd_but_z3_s0lv3r_1s_3asy}
[*] 正在提交: moectf{l@mbd4_c@lcu1us_4r3_h4rd_but_z3_s0lv3r_1s_3asy}
已提交，正在等待评测...
[+] 正确: Correct!
sunlight_tr4vel3r:sunlight_traveler ~/Mewtype — in MoeCTF 2026 [18:27:51]
$
```

#实际做了两个小时这玩意，回来写wp又看了半小时，晕了

Ultato Polosion Xptra

没做出来ww，先放着

请fanchai喝茶

老办法很爽。

```
__int64 __fastcall main(int a1, char **a2, char **a3)
{
    char *v3; // rsi
    __m128i *v4; // rdi
    __int64 v5; // rcx
    unsigned int v6; // edx
    unsigned __int32 v7; // esi
    unsigned __int32 v8; // ebp
    unsigned __int32 v9; // ebx
    unsigned __int32 v10; // r11d
    unsigned __int32 v11; // r10d
    unsigned __int32 v12; // r9d
    unsigned __int32 v13; // r8d
    unsigned __int32 v14; // edi
    __m128i v15; // xmm0
    __m128i v16; // xmm0
    unsigned __int32 v18; // [rsp+0h] [rbp-E8h]
    unsigned __int32 v19; // [rsp+4h] [rbp-E4h]
    unsigned __int32 v20; // [rsp+8h] [rbp-E0h]
    unsigned __int32 v21; // [rsp+Ch] [rbp-DCh]
    unsigned __int32 v22; // [rsp+10h] [rbp-D8h]
    unsigned __int32 v23; // [rsp+14h] [rbp-D4h]
    unsigned __int32 v24; // [rsp+18h] [rbp-D0h]
    unsigned __int32 v25; // [rsp+1Ch] [rbp-CCCh]
    __m128i v26; // [rsp+20h] [rbp-C8h] BYREF
    __m128i v27; // [rsp+30h] [rbp-B8h]
    __m128i v28; // [rsp+40h] [rbp-A8h]
    __m128i v29; // [rsp+50h] [rbp-98h]
    char s[136]; // [rsp+60h] [rbp-88h] BYREF
```

```

fwrite("Input flag: ", 1uLL, 0xCuLL, stdout);
fflush(stdout);
if ( !fgets(s, 66, stdin) )
    goto LABEL_9;
s[strcspn(s, "\r\n")] = 0;
if ( strlen(s) != 64 )
    goto LABEL_9;
v3 = s;
v4 = &v26;
v5 = 16LL;
v6 = 0;
while ( v5 )
{
    v4->m128i_i32[0] = *(_DWORD *)v3;
    v3 += 4;
    v4 = (__m128i *)((char *)v4 + 4);
    --v5;
}
v7 = v29.m128i_u32[3];
v8 = v28.m128i_i32[0];
v9 = v28.m128i_u32[1];
v10 = v28.m128i_u32[2];
v11 = v28.m128i_u32[3];
v25 = v26.m128i_i32[0];
v12 = v29.m128i_i32[0];
v13 = v29.m128i_u32[1];
v14 = v29.m128i_u32[2];
v20 = v26.m128i_u32[2];
v24 = v26.m128i_u32[1];
v19 = v26.m128i_u32[3];
v23 = v27.m128i_u32[1];
v18 = v27.m128i_i32[0];
v21 = v27.m128i_u32[3];
v22 = v27.m128i_u32[2];
do
{
    v6 += 1597463007;
    v25 += ((v6 ^ v24) + (v7 ^ dword_2020[(v6 >> 2) & 3])) ^ (((4 * v24) ^ (v7
>> 5)) + ((16 * v7) ^ (v24 >> 3)));
    v24 += (((16 * v25) ^ (v20 >> 3)) + ((4 * v20) ^ (v25 >> 5))) ^ ((v6 ^ v20)
+ (v25 ^
dword_2020[(((unsigned __int8)(v6 >> 2) ^ 1) & 3]));
    v20 += (((16 * v24) ^ (v19 >> 3)) + ((4 * v19) ^ (v24 >> 5))) ^ ((v6 ^ v19)
+ (v24 ^
dword_2020[(((unsigned __int8)(v6 >> 2) ^ 2) & 3]));
    v19 += (((16 * v20) ^ (v18 >> 3)) + ((4 * v18) ^ (v20 >> 5))) ^ ((v6 ^ v18)
+ (v20 ^
dword_2020[~((unsigned __int8)(v6 >> 2) & 3]));
    v18 += (((16 * v19) ^ (v23 >> 3)) + ((4 * v23) ^ (v19 >> 5))) ^ ((v6 ^ v23)
+ (v19 ^ dword_2020[(v6 >> 2) & 3]));
    v23 += (((16 * v18) ^ (v22 >> 3)) + ((4 * v22) ^ (v18 >> 5))) ^ ((v6 ^ v22)
+ (v18 ^
dword_2020[(((unsigned __int8)(v6 >> 2) ^ 5) & 3]));
    v22 += (((16 * v23) ^ (v21 >> 3)) + ((4 * v21) ^ (v23 >> 5))) ^ ((v6 ^ v21)
+ (v23 ^
dword_2020[(((unsigned __int8)(v6 >> 2) ^ 6) & 3]));
    v21 += (((16 * v22) ^ (v8 >> 3)) + ((4 * v8) ^ (v22 >> 5))) ^ ((v8 ^ v6)

```

```

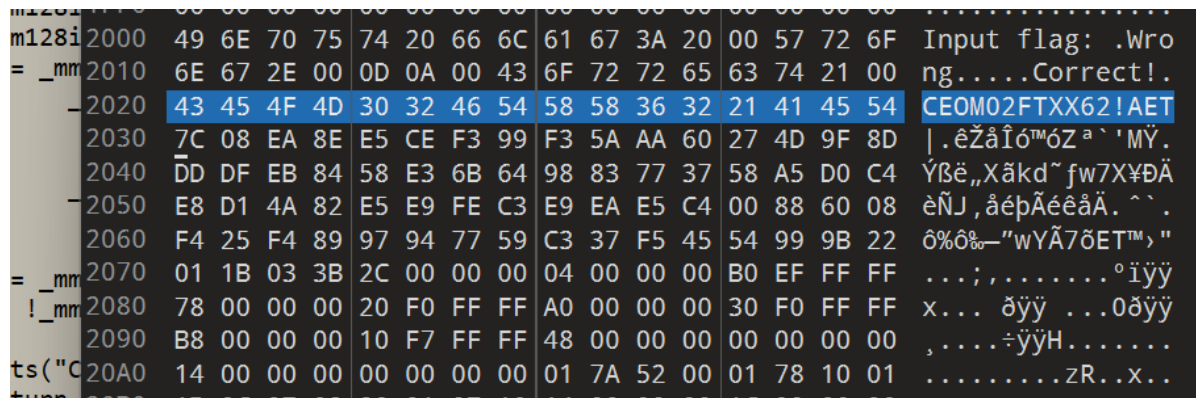
+ (v22 ^
dword_2020[((unsigned __int8)(v6 >> 2) ^ 7) & 3]));
v8 += (((16 * v21) ^ (v9 >> 3)) + ((4 * v9) ^ (v21 >> 5))) ^ ((v9 ^ v6) +
(v21 ^ dword_2020[(v6 >> 2) & 3]));
v9 += (((16 * v8) ^ (v10 >> 3)) + ((4 * v10) ^ (v8 >> 5))) ^ ((v10 ^ v6)
+ (v8 ^
dword_2020[((unsigned __int8)(v6 >> 2) ^ 9) & 3]));
v10 += (((16 * v9) ^ (v11 >> 3)) + ((4 * v11) ^ (v9 >> 5))) ^ ((v11 ^ v6)
+ (v9 ^
dword_2020[((unsigned __int8)(v6 >> 2) ^ 0xA) & 3]));
v11 += (((16 * v10) ^ (v12 >> 3)) + ((4 * v12) ^ (v10 >> 5))) ^ ((v12 ^ v6)
+ (v10 ^
dword_2020[((unsigned __int8)(v6 >> 2) ^ 0xB) & 3]));
v12 += (((16 * v11) ^ (v13 >> 3)) + ((4 * v13) ^ (v11 >> 5))) ^ ((v13 ^ v6)
+ (v11 ^ dword_2020[(v6 >> 2) & 3]));
v13 += (((v12 >> 5) ^ (4 * v14)) + ((v14 >> 3) ^ (16 * v12))) ^ ((v14 ^ v6)
+ (v12 ^
dword_2020[((unsigned __int8)(v6 >> 2) ^ 0xD) & 3]));
v14 += (((v13 >> 5) ^ (4 * v7)) + ((v7 >> 3) ^ (16 * v13))) ^ ((v7 ^ v6)
+ (v13 ^
dword_2020[((unsigned __int8)(v6 >> 2) ^ 0xE) & 3]));
v7 += (((16 * v14) ^ (v25 >> 3)) + ((4 * v25) ^ (v14 >> 5))) ^ ((v6 ^ v25)
+ (v14 ^
dword_2020[((unsigned __int8)(v6 >> 2) ^ 0xF) & 3]));
}
while ( v6 != 1492265175 );
v28.m128i_i64[0] = __PAIR64__(v9, v8);
v26.m128i_i64[0] = __PAIR64__(v24, v25);
v28.m128i_i64[1] = __PAIR64__(v11, v10);
v26.m128i_i64[1] = __PAIR64__(v19, v20);
v29.m128i_i64[0] = __PAIR64__(v13, v12);
v27.m128i_i64[0] = __PAIR64__(v23, v18);
v29.m128i_i64[1] = __PAIR64__(v7, v14);
v27.m128i_i64[1] = __PAIR64__(v21, v22);
v15 = _mm_or_si128(
    _mm_or_si128(
        _mm_xor_si128(_mm_load_si128((const __m128i *)&xmmword_2030), v27),
        _mm_xor_si128(_mm_load_si128((const __m128i *)&xmmword_2040), v26)),
    _mm_or_si128(
        _mm_xor_si128(_mm_load_si128((const __m128i *)&xmmword_2050), v29),
        _mm_xor_si128(_mm_load_si128((const __m128i *)&xmmword_2060),
v28)));
v16 = _mm_or_si128(v15, _mm_srli_si128(v15, 8));
if ( !_mm_cvtsi128_si32(_mm_or_si128(v16, _mm_srli_si128(v16, 4))) )
{
    puts("Correct!");
    return 0LL;
}
else
{
    LABEL_9:
    puts("Wrong.");
    return 1LL;
}
}

```

tea加密我们之前见识过了，但是这次我们尝试自己写解密脚本。

现在我们需要xmmword_2030-2060，以及dword_2020

直接用010Editor打开



ok了，那么dword_2020是43 45 4F 4D 30 32 46 54 58 58 36 32 21 41 45 54（密钥）

xmmword_2030是7C 08 EA 8E E5 CE F3 99 F3 5A AA 60 27 4D 9F 8D（密文）

xmmword_2040是DD DF EB 84 58 E3 6B 64 98 83 77 37 58 A5 D0 C4（密文）

xmmword_2050是E8 D1 4A 82 E5 E9 FE C3 E9 EA E5 C4 00 88 60 08（密文）

xmmword_2060是F4 25 F4 89 97 94 77 59 C3 37 F5 45 54 99 9B 22（密文）

（在ida中需手算，我们用010Editor省去计算）

那么我们写脚本吧！参照维基百科上tea解密的标准写法和先去tea解密的写法。

```
#include <stdio.h>
#include <stdint.h>
#include <string.h>

#define DELTA 0x5F3759DF

#define MX (((z>>5^y<<2) + (y>>3^z<<4)) ^ ((sum^y) + (key[(p&3)^e] ^ z)))

void btea_decrypt(uint32_t *v, int n, uint32_t const key[4]) {
    uint32_t y, z, sum;
    unsigned p, rounds, e;
    rounds = 6 + 52 / n;
    sum = rounds * DELTA;
    y = v[0];
    do {
        e = (sum >> 2) & 3;
        for (p = n - 1; p > 0; p--) {
            z = v[p - 1];
            y = v[p] -= MX;
        }
        z = v[n - 1];
        y = v[0] -= MX;
        sum -= DELTA;
    } while (--rounds);
}

int main() {
```

```

uint32_t key[4] = {
    0x4D4F4543,
    0x54463230,
    0x32365858,
    0x54454121
};

uint32_t v[16] = {
    // xmmword_2040
    0x84EBDFDD, 0x646BE358, 0x37778398, 0xc4D0A558,
    // xmmword_2030
    0x8EEA087C, 0x99F3CEE5, 0x60AA5AF3, 0x8D9F4D27,
    // xmmword_2060
    0x89F425F4, 0x59779497, 0x45F537C3, 0x229B9954,
    // xmmword_2050
    0x824AD1E8, 0xC3FEE9E5, 0xC4E5EAE9, 0x08608800
};

int n = 16;
btea_decrypt(v, n, key);

char flag[65];
for (int i = 0; i < 16; i++) {
    flag[i * 4 + 0] = (v[i] >> 0) & 0xFF;
    flag[i * 4 + 1] = (v[i] >> 8) & 0xFF;
    flag[i * 4 + 2] = (v[i] >> 16) & 0xFF;
    flag[i * 4 + 3] = (v[i] >> 24) & 0xFF;
}
flag[64] = '\0';

printf("Flag: %s\n", flag);
return 0;
}

```

那么运行，结束

flag:

moectf{Tre4t_f4ncha1_W1tH_xxtea_Wh4t_A_g00d_Id3a_HahaHAh4hAH4ha}

```

sunlight_tr4vel3r:sunlight_traveler ~/请 fanchai 喝茶 — in MoeCTF 2026 [21:52:09]
$ moectf{Tre4t_f4ncha1_W1tH_xxtea_Wh4t_A_g00d_Id3a_HahaHAh4hAH4ha}
[*] 正在提交: moectf{Tre4t_f4ncha1_W1tH_xxtea_Wh4t_A_g00d_Id3a_HahaHAh4hAH4ha}
已提交，正在等待评测...
[+] 正确: Correct!
sunlight_tr4vel3r:sunlight_traveler ~/请 fanchai 喝茶 — in MoeCTF 2026 [21:55:56]
$

```

持续更新ing.....敬请期待。

